

Modül 20

Server-Side Rendering

SSR + Hydration + Streaming

Modül:	20 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 20'ye Hoş Geldin

SSR — Server-Side Rendering. SEO, performance ve initial load için kritik. Modern Angular'da hydration, SSG (Static Site Generation), ve streaming SSR destekli.

Öğrenecekler:

- ✓ SSR vs CSR vs SSG karşılaştırma
- ✓ Angular SSR setup
- ✓ Hydration nedir, nasıl çalışır
- ✓ Platform check (isPlatformBrowser/Server)
- ✓ TransferState — server'dan client'a state
- ✓ Streaming SSR (v19+)
- ✓ SSR debugging ve common pitfalls

SSR, CSR, SSG — Karşılaştırma

Üç farklı rendering yaklaşımı.

Karşılaştırma Tablosu

Tür	Render Nerede	Use Case
CSR	Browser	SPA, dashboard, app
SSR	Server (her request)	SEO, dynamic content
SSG	Build time (statik)	Blog, marketing, docs
ISR	Server (cache)	E-commerce

CSR (Client-Side Rendering)

```
Request → Server: HTML iskelet (sadece <app-root></app-root>)
      ↓
Browser: index.html parse
      ↓
Browser: JS yükle
      ↓
Browser: Angular bootstrap
      ↓
Browser: Component render
      ↓
Browser: HTTP istekleri
      ↓
İçerik görünür (FCP geç)
```

SSR

```
Request → Server: Angular render et
      ↓
Server: Tam HTML
      ↓
Browser: HTML görüntülenir (FCP hızlı!!)
      ↓
Browser: JS yüklenir (arka planda)
      ↓
Browser: Hydration (event listener'lar)
      ↓
Interactive
```

□ SSG

Build time:

ng build --prerender

Server: Tüm sayfaları HTML olarak üret

↓

Dist klasörü: index.html, blog/post-1.html, blog/post-2.html...

Runtime:

Request → CDN: Statik HTML hemen döner (en hızlı)

↓

Hydration (gerekirse)

SSR Setup

Angular projesine SSR ekleme.

⚙️ Yeni Proje SSR ile

```
ng new my-app --ssr

# Var olan projeye ekle
ng add @angular/ssr
```

📁 Dosya Yapısı

```
src/
  app/
    app.component.ts
    app.routes.ts
    app.config.ts           # Browser config
    app.config.server.ts   # Server config
  main.ts                  # Browser entry
  main.server.ts           # Server entry
  server.ts                # Express server

# Build edince:
dist/my-app/
  browser/                 # Client bundle
  server/                  # Server bundle
```

⚡ app.config.server.ts

```
import { mergeApplicationConfig, ApplicationConfig } from '@angular/core';
import { provideServerRendering } from '@angular/platform-server';
import { appConfig } from './app.config';

const serverConfig: ApplicationConfig = {
  providers: [
    provideServerRendering(),
  ]
};

export const config = mergeApplicationConfig(appConfig, serverConfig);
```

📁 Çalıştırma

```
# Development  
npm run dev:ssr
```

```
# Production build  
npm run build
```

```
# Production server  
npm run serve:ssr
```

Hydration

Server-rendered HTML'i interactive yapma.

❏ Hydration Nedir?

Server statik HTML gönderir. Bu HTML'i event listener'lar ve state ile "canlandırmaya" hydration denir.

⚙️ Hydration Setup

```
import { provideClientHydration } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideClientHydration(), // Otomatik aktif
  ]
};
```

❏ Faydaları

- ✓ Flicker yok: Eski yaklaşımda content destroy → rebuild
- ✓ Hızlı interactive: Mevcut DOM'u kullan, yeniden create etme
- ✓ State preservation: Form input'ları kaybedilmez
- ✓ Better performance: %50+ daha az network/DOM operations

❏ Incremental Hydration (v19+)

```
// Sadece kullanıcı etkileşince hydrate
@defer (hydrate on viewport) {
  <app-comments />
}

@defer (hydrate on interaction) {
  <app-heavy-widget />
}

@defer (hydrate never) {
  <app-static-content /> // Asla JS yüklenmesin
}
```

Platform Check

Server'da çalışmaması gereken kod'u korumak.

□ isPlatformBrowser/Server

```
import { Component, inject, PLATFORM_ID } from '@angular/core';
import { isPlatformBrowser, isPlatformServer } from '@angular/common';

@Component({...})
export class MyComponent {
  private platformId = inject(PLATFORM_ID);

  constructor() {
    if (isPlatformBrowser(this.platformId)) {
      // Sadece browser'da çalışsın
      window.addEventListener('scroll', this.onScroll);
      localStorage.setItem('lastVisit', new Date().toISOString());
    }

    if (isPlatformServer(this.platformId)) {
      // Sadece server'da
      console.log('Rendering on server');
    }
  }
}
```

□ Yaygın Hatalar

- ✗ window/document erişimi — server'da yok, hata verir
- ✗ localStorage/sessionStorage — sadece browser
- ✗ navigator objesi — server'da farklı
- ✗ setTimeout/setInterval — temizlemek lazım yoksa SSR hangs
- ✗ document.createElement — Renderer2 kullan

□ Safe Patterns


```
// ✓ Renderer2 – platform-agnostic
import { Renderer2 } from '@angular/core';

@Component({...})
export class MyComponent {
  private renderer = inject(Renderer2);

  setBackground() {
    this.renderer.setStyle(this.el.nativeElement, 'background', 'red');
    // Browser'da: DOM API
    // Server'da: HTML string
  }
}

// ✓ afterNextRender – sadece browser'da çalışır
import { afterNextRender } from '@angular/core';

constructor() {
  afterNextRender(() => {
    // Server'da otomatik atlanır
    window.scrollTo(0, 0);
  });
}
```

TransferState

Server'da yapılan HTTP isteklerini client'ta tekrar yapma!

❑ Problem

SSR'da: Server HTTP isteği yapar → HTML render eder

Client hydrate: Aynı HTTP isteği tekrar yapılır → boşa istek

❑ Çözüm: TransferState

Modern Angular HttpClient ile otomatik. Sadece `provideClientHydration(withHttpTransferCacheOptions())` ekle.

```
import { provideClientHydration, withHttpTransferCacheOptions } from
 '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideClientHydration(
      withHttpTransferCacheOptions({
        includePostRequests: false, // Default: false
        filter: req => req.url.startsWith('/api/'), // Hangi istekler cache
      })
    ),
    provideHttpClient(withFetch()),
  ]
};
```

❑ Manual TransferState

```
import { TransferState, makeStateKey } from '@angular/core';

const USERS_KEY = makeStateKey<User[]>('users');

@Injectable({...})
export class UserService {
  private state = inject(TransferState);
  private http = inject(HttpClient);
  private platformId = inject(PLATFORM_ID);

  getUsers() {
    // Server'da yapıldıysa
    const cached = this.state.get(USERS_KEY, null);
    if (cached) {
      this.state.remove(USERS_KEY);
      return of(cached);
    }

    return this.http.get<User[]>('/api/users').pipe(
      tap(users => {
        if (isPlatformServer(this.platformId)) {
          this.state.set(USERS_KEY, users);
        }
      })
    );
  }
}
```

Streaming SSR (v19+)

HTML'i parça parça stream et — daha hızlı first byte.

□ Streaming Nedir?

Klasik SSR: Tüm sayfa server'da render edilir, sonra browser'a gönderilir.

Streaming SSR: Hazır olan parçalar hemen gönderilir, hala render edilen parça geldikçe gönderilir.

⚡ @defer (hydrate)

```
<header>
  <h1>{{ title() }}</h1> <!-- Hemen render -->
</header>

@defer (hydrate on viewport) {
  <app-comments /> <!-- Lazy hydrate -->
}

@defer (on viewport) {
  <app-heavy-chart /> <!-- Lazy load + lazy hydrate -->
}
```

□ Use Cases

- ✓ Above-the-fold önce gelsin, alt parçalar sonra
- ✓ 3rd party widget bekleyen sayfa hızlı görünsün
- ✓ Heavy components: chart, map, video player

Debugging & Common Pitfalls

SSR'da en sık karşılaşılan sorunlar.

☐ Hydration Mismatch

⚠ Hydration Mismatch

Server'ın render ettiği HTML ile client'ın beklediği HTML farklı olursa, Angular warning veya error verir. Çözüm: aynı koşullarda aynı output üretmek.

☐ Common Mistakes

- ✗ `new Date()` — server ve client farklı zamanlar
- ✗ `Math.random()` — server ve client farklı sonuçlar
- ✗ `window.innerWidth` — server'da yok
- ✗ `localStorage` okuyarak conditional render
- ✗ 3rd party libs `document/window`'a dokunuyorsa

☐ Solutions

```
// ☐ Hydration mismatch
@Component({
  template: `<p>{{ now() }}</p>`
})
export class Bad {
  now = signal(new Date().toLocaleString());
  // Server: 14:35:00, Client: 14:35:02 → mismatch
}

// ✓ afterNextRender kullan
@Component({
  template: `<p>{{ now() }}</p>`
})
export class Good {
  now = signal('');

  constructor() {
    afterNextRender(() => {
      this.now.set(new Date().toLocaleString());
    });
  }
}
```

🔧 Debugging Tools

- ✓ ngDevMode warnings'leri kontrol et
- ✓ Browser DevTools → Disable JS, sayfa hala çalışmalı (SSR proof)
- ✓ View Source → HTML tam render edilmiş mi
- ✓ Network tab → JS yüklenmeden FCP olmuş mu

⚙️ Production Checklist

- ✓ provideClientHydration aktif
- ✓ HTTP transfer cache ayarlanmış
- ✓ SEO meta tags her sayfada
- ✓ Sitemap generate edilmiş
- ✓ robots.txt ayarlanmış
- ✓ Server'da memory leak testi yapılmış (long-running)

Modül 20 Özeti

SSR konusunda artık bilinçli kararlar verebilirsin. Hydration, TransferState, streaming — modern Angular SSR'ın gücü.

Neler Öğrendin?

- ✓ SSR, CSR, SSG farkları
- ✓ Angular SSR setup
- ✓ Hydration ve faydaları
- ✓ Platform check (isPlatformBrowser/Server)
- ✓ TransferState ile HTTP cache
- ✓ Streaming SSR ve @defer hydrate
- ✓ Common pitfalls ve debugging

Sıradaki Modül

Modül 21 — Progressive Web Apps (PWA). Offline support, install, push notifications, service worker.